Block 4 – Session 1

Data Centers & Network Virtualization

Arquitectura de Xarxes

Universitat Pompeu Fabra

Adrián Pino <adrian.pino@upf.edu>

upf.

Outline

upf.

What Is the Cloud and How Is It Formed?

Every time you use Netflix, Gmail, or ChatGPT,
your request travels to a building full of servers.
What happens inside that building?

Learning objectives:

1. Understand why data centers exist and what they contain
2. Explain virtualization and distinguish VMs, hypervisors, and containers
3. Describe how networks are virtualized (vNICs, vSwitches, overlays)

upf.

Block 4 Roadmap

	Session 1 (today)	**Session 2**
Theme	*The infrastructure*	*The business model*
Topics	Data centers	Cloud & Container platforms
	VMs & hypervisors	Overlay networks (VXLAN)
	VMs vs containers (overview)	NIST cloud definition
	vNICs, vSwitches, vRouters	IaaS / PaaS / SaaS
	Multi-tenancy & VLANs	Security groups & ACLs

Outline

upf.

What Is a Data Center?

- A **data center** is a facility housing computer systems and networking equipment
- Purpose: run applications and store data **reliably, at scale, 24/7**
- Ranges from small server rooms to **warehouse-scale** buildings

Who operates them?

- **Enterprises**: banks, hospitals, universities (their own IT)
- **Cloud providers**: AWS, Azure, GCP (sell capacity to others)
- **Colocation**: you rent space; provider handles power and cooling

Source: Barroso, Clidaras & Hölzle, *The Datacenter as a Computer*, 3rd ed., Morgan & Claypool, 2019.

upf.

A Data Center from the Inside

Rows of server racks, each containing dozens of servers, connected by structured cabling. Source: Cisco, *What Is a Data Center?*, cisco.com.

Inside a Data Center: Five Pillars

- **Compute**: CPUs that execute workloads (web servers, databases, analytics)
- **Accelerators**: GPUs and TPUs for AI/ML training and inference
  - The explosive growth of AI has made GPUs a critical data center resource
- **Storage**: SSDs and HDDs holding persistent data
- **Network**: switches, routers, and structured cabling connecting everything
- **Facilities**: redundant power supplies, cooling systems, physical security

The Problem with Physical Infrastructure

- Traditional model (1990s–early 2000s): **one server = one application**
- Typical server utilization: only **10–15%** of capacity
- Adding capacity means buying, racking, cabling $\rightarrow$ **weeks or months**
- Hardware failure takes down the entire service
- No way to scale down when demand drops

Key insight: most servers sit idle most of the time. We are paying for hardware we barely use.

upf.

The Cloud: From Waste to Efficiency

- **1990s–early 2000s**: one application per server; most capacity wasted
- **2001**: VMware makes it possible to run **multiple systems on one machine**
  - Technology: **Virtual Machines (VMs)**
- **Mid-2000s**: enterprises consolidate servers (60–80% utilization)
- **2006–2010**: Amazon, Google, and Microsoft start **renting computing capacity** online
  - Technology: **Cloud Computing** (AWS, Azure, GCP)
- **2010s–today**: lightweight alternatives emerge; automation at massive scale
  - Technologies: **Containers** (Docker) and **orchestration** (Kubernetes)

We will explore each of these technologies step by step throughout Blocks 4 and 5.

upf.

Scalability and Elasticity

- **Scalability**: ability to grow resources as demand increases
- **Elasticity**: ability to grow *and shrink* automatically
- Physical infrastructure provides **neither** in practice:
  - Cannot add servers in minutes
  - Cannot return servers when demand drops
- Virtualization enables both $\rightarrow$ foundation of **cloud computing** (Session 2)

upf.

Discussion: Data Centers

A company runs 50 servers, each at 10 percent utilization.
That is 50 machines doing the work of 5.
What would you do to reduce waste?

Hints: consolidation, fewer physical machines, higher utilization, isolation between workloads.

Outline

upf.

What Is Virtualization?

- **Virtualization** makes it possible for a single computer to act like many
- A software layer called **hypervisor** divides the physical resources (CPU, RAM, disk, Network Interface Card (NIC)) into isolated **virtual machines (VMs)**
- Each VM gets its own share and believes it owns dedicated hardware

Why virtualize?

- **Resource efficiency**: consolidate workloads instead of one app per server
- **Cost savings**: fewer servers, less power, cooling, and space
- **Flexibility**: create or destroy environments in minutes
- **Isolation**: VMs are independent; failures do not propagate

upf.

Virtual Machines (VMs)

- A VM is an **isolated computing environment** with its own CPU, memory, network interface, and storage
- Runs a **full operating system** (Linux-based OSs such as Ubuntu, Windows, etc.)
- Completely **isolated** from other VMs on the same host

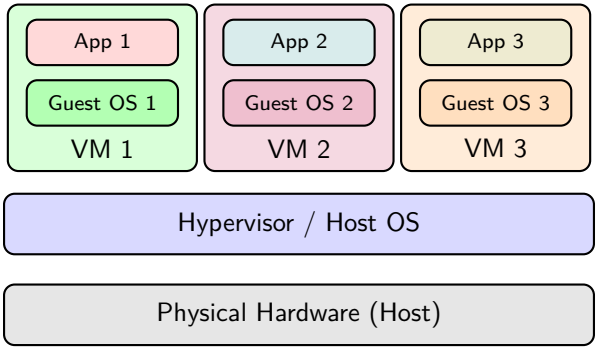
Key properties:

- **Encapsulation**: entire VM state stored as files → easy to copy, move, backup
- **Hardware independence**: the VM does not care about the underlying hardware
- **Snapshotting**: save and restore VM state at any point in time

Source: Popek & Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures," *CACM*, 1974.

upf.

Host and Guest Systems



- **Host**: physical machine providing CPU, RAM, disk, NIC
- **Guest**: virtual machine running its own OS on the host
- Each guest believes it has **dedicated hardware** (abstraction)

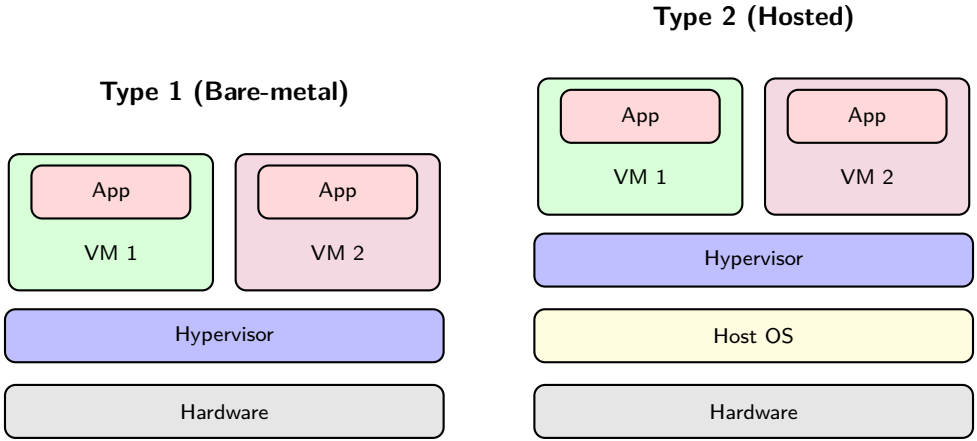
Hypervisors: The Key Enabler

- A **hypervisor** manages and allocates physical resources to VMs
- Two types:

	Type 1 (Bare-metal)	**Type 2 (Hosted)**
Runs on	Directly on hardware	On top of a host OS
Performance	Near-native	Some overhead
Use case	Data centers, production	Development, testing
Examples	VMware ESXi, KVM (Kernel-based VM)	VirtualBox, VMware Workstation

Source: Barham et al., "Xen and the Art of Virtualization," *SOSP*, 2003.

Hypervisors Type 1 vs Type 2: Visual Comparison



- Type 1: hypervisor runs **directly on hardware** (no host OS) → less overhead (faster)
- Type 2: hypervisor runs **on top of a host OS** → easier to set up

Discussion: Virtualization Concepts

You want to learn about virtualization at home.
Would you install a Type 1 or a Type 2 hypervisor?
What if you were setting up a production server?

Hints: Type 2 runs on your existing OS (easy to try); Type 1 replaces the OS entirely (better performance, but dedicated hardware).

Outline

upf.

What Is a Container?

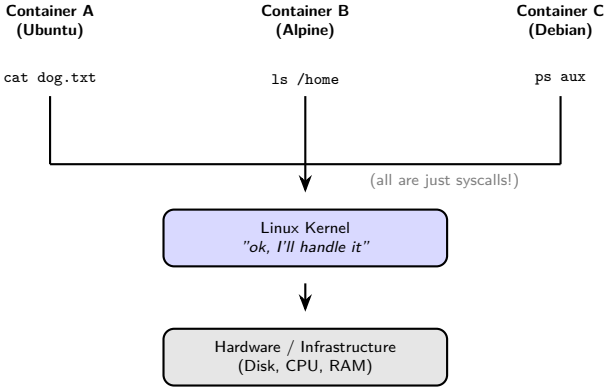
- A **container** is a lightweight, isolated environment that runs an application
- Unlike a VM, it does **not** include a full operating system
  - It shares the **host's kernel** and only packages the app + its dependencies

- Key characteristics:
  - **Fast**: starts in seconds (no OS to boot)
  - **Small**: measured in MBs, not GBs
  - **Portable**: same container runs identically on any machine with a container runtime
  - **Ephemeral**: designed to be created, destroyed, and replaced quickly

The most popular container engine is **Docker** (2013). Container orchestration (managing thousands of containers) is handled by **Kubernetes** (2014). Deep dive in **Block 5**.

upf.

Containers Share the Same... Kernel?



- Every command inside any container is a **syscall** to the one shared kernel.
- The kernel uses **linux namespaces** to ensure each container only sees its own files, processes, and network.

upf.

VMs vs Containers: Overview

Virtual Machines

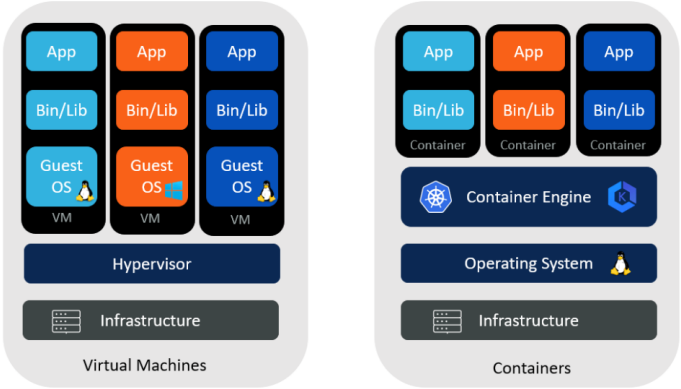
- Include a **full guest OS** per instance
- Size: **GBs** (OS + app + dependencies)
- Boot time: usually **minutes**
- Each VM has its own kernel; a compromised VM cannot affect others
- Can run **different operating systems**

Containers

- **No guest OS**; share the host kernel
- Size: **MBs** (app + dependencies only)
- Boot time: usually **seconds**
- Shared kernel means a kernel vulnerability can affect all containers
- No hypervisor needed to run. Instead a container engine/runtime is required

Reminder: OS = Kernel + userspace (libraries, tools, shell, etc.). Deep dive into container internals and networking in **Block 5**.

VMs vs Containers: Visual Comparison



- Containers share the same Kernel
- **Bins/Libs** = binaries and libraries: the dependencies each app needs to run (e.g., Python, OpenSSL, libc).

Source: Aqueduct Technologies, "Containers and Virtual Machines," aqueducttech.com.

When to Use VMs vs Containers

- **Use VMs when** the app requires it:
  - Needs a **different OS** than the host (e.g., Windows app on a Linux host)
  - Requires **strong tenant isolation** (e.g., multi-tenant cloud)
  - **Legacy software** tied to a specific OS version

- **Use containers when** the app allows it:
  - Cloud-native or **microservices** applications
  - Need for **fast scaling** (seconds, not minutes)
  - **CI/CD (Continuous Integration / Continuous Deployment) pipelines**: build, test, deploy in identical environments

- **Evolution**: infrastructure has evolved from pure VMs to today's hybrid VM+container model
  - Most new applications are designed for containers
  - VMs remain for legacy workloads and strong isolation
  - The future points toward a **container-first** approach, where VMs become the exception

upf.

Discussion: Virtualization

You need to deploy 200 instances of the same web application that must scale up and down every few minutes. Would you use VMs, containers, or both? Why?

Hints: boot time, resource overhead, isolation needs, how fast you need to scale.

upf.

Outline

upf.

From Physical to Virtual Networks

- You already know how physical networks work (Blocks 1 to 3):
  - NICs, switches, routers, VLANs, routing protocols
- When we virtualize servers, we also need to **virtualize the network**
- The same concepts apply, but implemented **in software** instead of hardware

Physical	**Virtual equivalent**
NIC (network interface card)	**Virtual NIC** (vNIC)
Switch	**Virtual Switch** (bridge)
Router	**Virtual Router**
Cable	Internal software path

Note: This section focuses on VMs. Containers follow similar principles (Block 5).

upf.

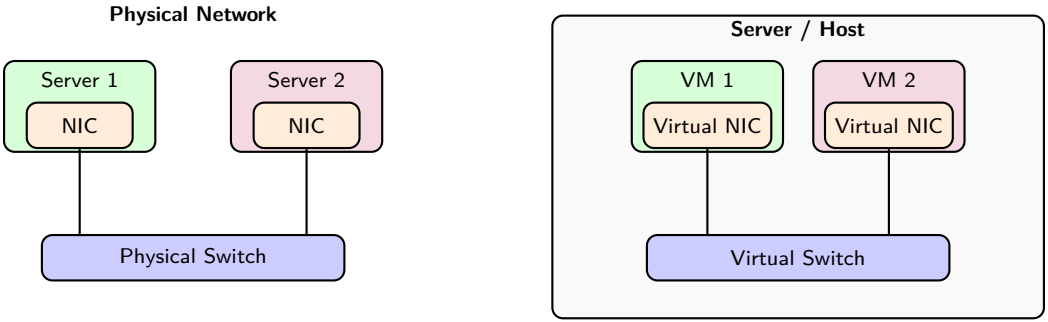
Virtual NIC, Virtual Switch, Virtual Router

- **Virtual NIC (vNIC)**: software-emulated network interface card
  - Has its own **MAC address** (assigned by the hypervisor) and **IP address**
  - From the guest's perspective, behaves exactly like a real NIC
- **Virtual Switch**: connects VMs at **L2** (same subnet)
  - Ranges from simple (Linux bridge) to programmable (Open vSwitch)
  - We will explore SDN-based switches in Block 5
- **Virtual Router**: forwards traffic between **different subnets** (L3)
  - Acts as **default gateway** for VMs
  - Can apply **NAT** for external access

Same MAC/IP/NAT concepts from Blocks 1–3, now virtualized inside the hypervisor.

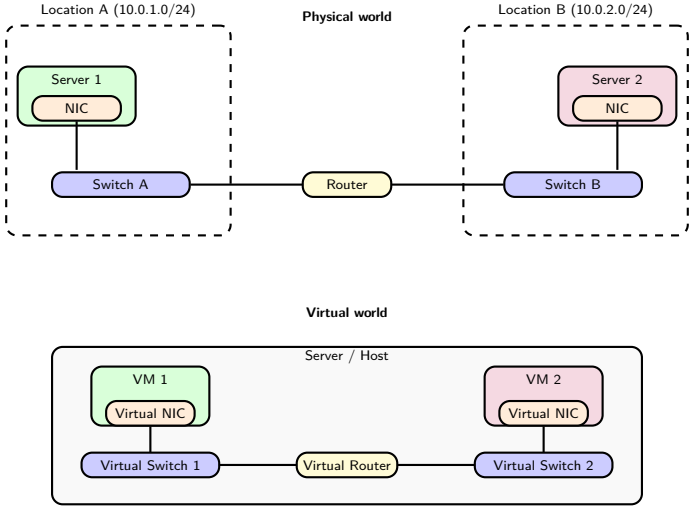
upf.

Physical vs Virtual: Side by Side (Same Network)



- Left: two physical servers connected by a physical switch and cables
- Right: two VMs connected by a virtual switch, all inside **one physical host**

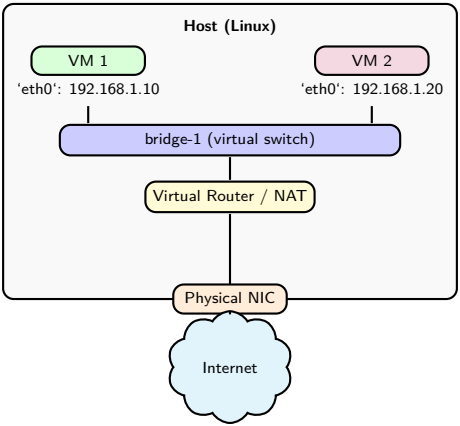
Physical vs Virtual: Side by Side (Different Networks)



Different subnets → router needed (Layer 3). Physical uses hardware; virtual uses software inside one host.

Example: VM-to-VM Communication

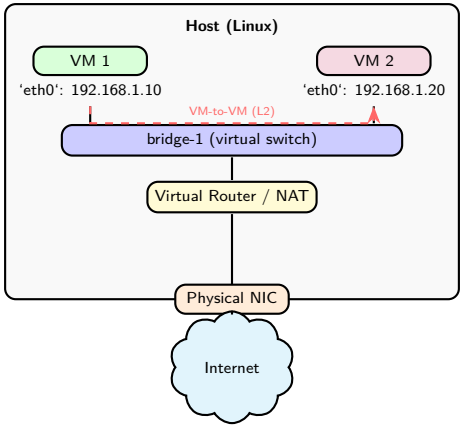
- The hypervisor creates a **bridge** (a virtual switch) to connect VMs
- VMs on the same bridge share a subnet: frames forwarded by MAC address



How does VM 1 reach VM 2? And how does it reach the Internet?

Example: VM-to-VM Communication (Solution)

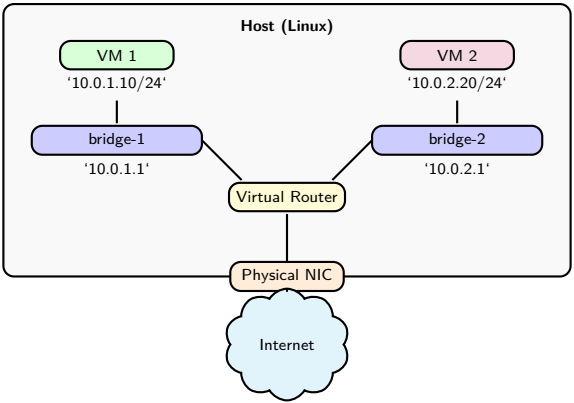
- Same subnet, same bridge → **Layer 2 (L2) forwarding** (no routing needed)



Both VMs share bridge-1: it forwards frames by MAC address, just like a physical switch. No routing needed.

Example: VMs on Different Subnets

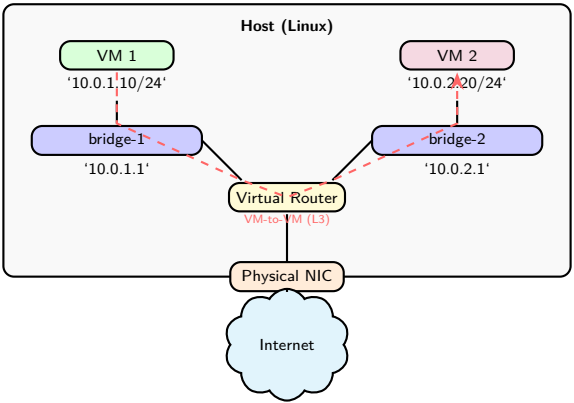
- Now VMs are on **different subnets**: they need Layer 3 (L3) routing
- Two bridges (`bridge-1`, `bridge-2`), each with its own subnet
- Linux can act as a **virtual router** by default: no extra software needed



How does VM 1 (10.0.1.10) reach VM 2 (10.0.2.20)? What components are involved?

Example: VMs on Different Subnets (Solution)

- Different subnets → traffic must go through the **virtual router** (L3 forwarding)



Each VM has a default gateway pointing to its bridge IP. The host routes packets between subnets, just like a physical router.

VM Networking Modes

How does a VM connect to the outside world? Three modes:

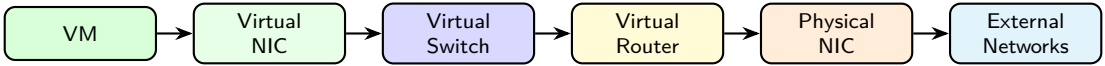
1. **NAT** (most common): VM hides behind the host's IP
   - VM can reach the Internet, but the Internet cannot reach the VM
   - Same NAT concept from Block 3, now applied by the hypervisor
2. **Bridged**: VM appears directly on the physical network
   - Same subnet as the host; gets its own IP from the network
3. **Host-only**: VM can only talk to the host
   - Completely isolated from external networks; useful for testing

NAT is the default in most hypervisors (VirtualBox, VMware, KVM). Bridged and Host-only exist for specific use cases.

In Session 2 we will see how cloud providers offer NAT as a **managed service** (NAT Gateway).

upf.

Packet Flow: Virtual to Physical



1. VM generates packet with **virtual MAC/IP** as source
2. Virtual NIC passes frame to **Virtual Switch** (L2 forwarding)
3. If destination is another subnet $\rightarrow$ **Virtual Router** (L3 forwarding)
4. Virtual Router may apply **NAT** (replace private IP with host IP)
5. Frame exits through **Physical NIC** to external networks

Discussion: Virtual Networking

Two VMs on the same host want to communicate.
Does their traffic ever leave the physical machine?
What if they are on different subnets?

Hints: Virtual Switch handles same-subnet traffic locally; Virtual Router needed for different subnets; traffic may still stay inside the host.

upf.

Outline

upf.

The Problem: Shared Infrastructure, Private Data

- Cloud platforms host workloads from **multiple tenants** (users, departments, companies)
- All tenants share the same physical servers, switches, and cabling
- But each tenant expects:
  - **Security**: you cannot see or access my data
  - **Performance**: my traffic is not affected by yours
  - **Address independence**: we can both use `10.0.0.0/24` without conflicts

Cloud platforms solve this with **three layers of isolation**:

1. **VLANs**: separate traffic at the switch level (L2)
2. **Overlay networks**: encapsulate tenant traffic over the shared physical network (L2/L3)
3. **Micro-segmentation**: per-VM firewall rules via Security Groups and Access Control Lists (ACLs) (L3/L4)

We will explore overlay networks and cloud security in detail in **Session 2**.

Layer 1: VLANs (What You Already Know)

- Same VLAN concept from Block 3, now applied to **virtual switches**
- Hypervisors assign VMs to specific VLANs via virtual switch port groups
- Traffic tagged with VLAN IDs $\rightarrow$ tenants separated at **L2**

But there is a problem:

- VLAN ID field is 12 bits $\rightarrow$ max **4,096 VLANs**
- A large data center may have **tens of thousands** of tenants
- VLANs also do not support **overlapping IP ranges** across tenants

Source: IEEE 802.1Q-2022, "Bridges and Bridged Networks."

upf.

Layer 2: Overlay Networks

- Solution to the VLAN limit: **overlay networks** (e.g., Virtual Extensible LAN, or VXLAN)
  - Encapsulate tenant frames inside UDP packets over the physical network
  - 24-bit ID $\rightarrow$ up to **16 million** isolated networks
- Each tenant gets a fully isolated virtual network:
  - Own IP address space, subnets, and routing

Deep dive into VXLAN and overlay architecture in **Session 2**.

Layer 3: Micro-Segmentation

- VLANs and overlays isolate **networks**; micro-segmentation isolates **individual VMs**
- Goal: **least privilege**; each VM should only reach what it strictly needs
- Key concept in modern **zero trust** security architectures

Cloud platforms implement this with two mechanisms:

- **Security Groups**: firewall rules attached to each VM instance
- **Network ACLs**: firewall rules applied at the subnet level

Deep dive into Security Groups and Network ACLs in **Session 2**.

upf.

Isolation: The Full Picture

Layer	Mechanism	Scope
L2	**VLANs** (802.1Q)	Separate traffic at the switch level (max 4K)
L2/L3	**Overlays** (VXLAN)	Encapsulate tenant traffic (up to 16M networks)
L3/L4	**Security Groups / ACLs**	Per-VM and per-subnet firewall rules

Each layer adds isolation on top of the previous one. Today we covered VLANs in depth; overlays and security groups are the focus of **Session 2**.

Discussion: Network Isolation

You manage a cloud platform with 5,000 tenants.
Each tenant wants their own isolated network.
Can you use VLANs alone? Why or why not?

Hints: think about the VLAN ID limit (4,096), overlapping IP ranges, and what happens when VMs move between hosts.

Outline

upf.

Key Takeaways

1. **Data centers** house compute, accelerators (GPUs), storage, network, and facilities
2. Physical infrastructure is **rigid, underutilized, and slow to scale**
3. **Hypervisors** (Type 1 and 2) enable multiple VMs on one host
4. **Containers** are lighter than VMs but share the kernel (deep dive in Block 5)
5. Virtual networks mirror physical ones: **vNICs, vSwitches, vRouters**
6. **Multi-tenancy** requires isolation $\rightarrow$ VLANs and micro-segmentation

References

1. Popek & Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures," *CACM*, vol. 17, no. 7, 1974.
2. Barham et al., "Xen and the Art of Virtualization," *SOSP*, 2003.
3. VMware, "Understanding Full Virtualization, Paravirtualization, and Hardware Assist," 2007.
4. Pfaff et al., "The Design and Implementation of Open vSwitch," *NSDI*, 2015.
5. IEEE 802.1Q-2022, "Bridges and Bridged Networks."
6. Barroso, Clidaras & Hölzle, *The Datacenter as a Computer*, 3rd ed., Morgan & Claypool, 2019.
7. NIST SP 800-125, "Guide to Security for Full Virtualization Technologies," 2011.
8. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
9. Red Hat, "What is a Virtual Machine?", redhat.com/en/topics/virtualization/what-is-a-virtual-machine.
10. Red Hat, "Containers vs VMs," redhat.com/en/topics/containers/containers-vs-vms.
11. Cisco, "What Is a Data Center?", cisco.com/site/us/en/learn/topics/computing/what-is-a-data-center.html.

upf.